Table of Contents

15.Pre-requisites:.....1
15.1 Purpose:.....2
15.2 Introduction:.....3
1) 15.3 Detailed Explanation: 4
15.3.1 Annotation Elements: 4
15.3.2 Annotation Placement 5
15.3.3 Annotations Categories: 5
15.3.4 Exploring Simple Annotations 6
15.3.5 Meta Annotations 10

ANNOTATIONS

15.Pre-requisites:

If you look inside a JDK package (says java.lang), besides classes, interfaces, exceptions and errors, JDK 1.5 introduces two new entities: enums and annotations.

Annotation gives you the ability to provide additional metadata alongside a Java entity (such as classes, interfaces, fields and methods). This additional metadata, called annotation, can be read and interrelated by the compiler or other utilities. They can also be stored in the class files. The runtime can discovered these metadata via the "reflection" API.

Prior to the introduction of this standardized annotation feature in JDK 1.5, Java provided only ad-hoc and non-standardized mechanism, Annotation is a new feature introduced by J2SE 5 that allows programmers to embed additional information called metadata into a Java source file. Annotations do not alter the execution of a program but the information embedded using annotations can be used by various tools during development and deployment.

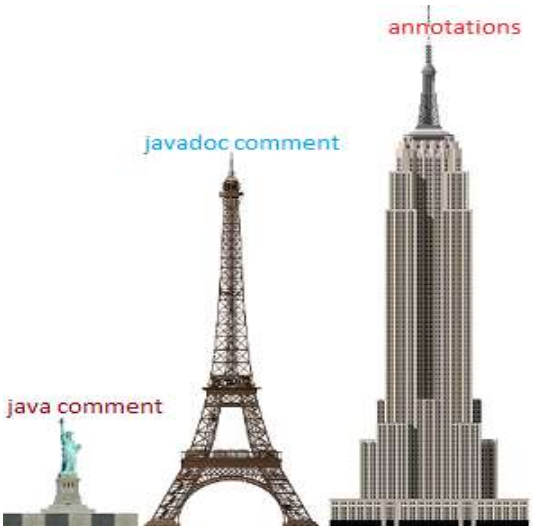
15.1 Purpose:

Annotation is code about the code, that is metadata about the program itself. In other words, organized data about the code, embedded within the code itself. It can be parsed by the compiler, annotation processing tools and can also be made available at run-time too.

One of the main reasons for adding annotation and metadata to the Java platform is to enable development and runtime tools to have a common infrastructure so as to reduce the effort required for development and deployment.

We have basic java comments infrastructure using which we add information about the code / logic so that in future, another programmer or the same programmer can understand the code in a better way. Javadoc is an additional step over it, where we add information about the class, methods, variables in the source code. The way we need to add is organized using a syntax. Therefore, we can use a tool and parse those comments and prepare a javadoc document which can be distributed separately.

Javadoc facility gives option for understanding the code in an external way, instead of opening the code the javadoc document can be used separately. IDE benefits using this javadoc as it is able to render information about the code as we develop. Annotations were introduced in JDK 1.5



Annotations are far more powerful than java comments and javadoc comments. One main difference with annotation is it can be carried over to runtime and the other two stops with compilation level. Annotations are not only comments, it brings in new possibilities in terms of automated processing.

Java annotations are typically used for the following purposes:

- Information for the compiler — Annotations can be used by the compiler to detect errors or suppress warnings.
- Compile-time and deployment-time processing — Software tools can process annotation information to generate code, XML files, and so forth.
- Runtime processing — Some annotations are available to be examined at runtime.

In java we have been passing information to compiler for long. For example take serialization, we have the keyword transient to tell that this field is not serializable. Now instead of having such keywords decorating an attribute annotations provide a generic way of adding information to class/method/field/variable. This is information is meant for programmers, automated tools, java compiler and runtime. Transient is a modifier and annotations are also a kind of modifiers.

15.2 Introduction:

Annotation is code about the code, that is metadata about the program itself. In other words, organized data about the code, embedded within the code itself. It can be parsed by the compiler, annotation processing tools and can also be made available at run-time too.

Annotation is a tag that represents the metadata. It is attached with class, interface, methods or fields to indicate some additional information that can be used by java compiler and JVM.

2) 15.3 Detailed Explanation:

Annotations, a form of metadata, provide data about a program that is not part of the program itself. Annotations have no direct effect on the operation of the code they annotate.

This feature was added to Java 5.0 as a result of the JSR 175 namely **"A Metadata Facility for the JavaTM Programming Language"**. Annotations in Java can be seen elsewhere in a program. It can be seen in class declaration, method declaration, field declaration etc. The added Annotation to Java Elements have proven themselves to be considerably useful in many instances.

An annotation in its shortest form looks like this: **@Entity**

The @ character signals to the compiler that this is an annotation. The name following the @ character is the name of the annotation. In the example above the annotation name is Entity.

Note: Frameworks like Hibernate, Spring, Axis make heavy use of annotations. When a language needs to be made popular one of the best thing to do is support development of frameworks based on the language. Annotation is a good step towards that and will help grow Java.

15.3.1 Annotation Elements:

Annotations can have elements for which you can set values. An element is like an attribute or parameter.

There are two main components in annotations. First is annotation type and the next is the annotation itself which we use in the code to add meaning. Every annotation belongs to a annotation type.

Annotation Type:

```
@Entity <annotation-type-name> {
method declaration;
}
```

The at sign character (@) indicates to the compiler that what follows is an annotation. In the following example, the annotation's name is Override:
Example:

```
@Override
void mySuperMethod() { ... }
```

The annotation can include elements, which can be named or unnamed, and there are values for those elements:

```
@SuppressWarnings(value = "unchecked")
void myMethod() { ... }
```

If there is just one element named value, then the name can be omitted, as in:

```
@SuppressWarnings("unchecked")
void myMethod() { ... }
```

If the annotation has no elements, then the parentheses can be omitted, as shown in the previous @Override example.

15.3.2 Annotation Placement

You can put Java annotations above classes, interfaces, methods, method parameters, fields and local variables. Here is an example annotation added above a class definition:

```
@Entity
public class ClassName{
}
```

The annotation starts with the @ character, followed by the name of the annotation. In this case, the annotation name is Entity.

15.3.3 Annotations Categories:

JDK 5 contains two categories of annotations:

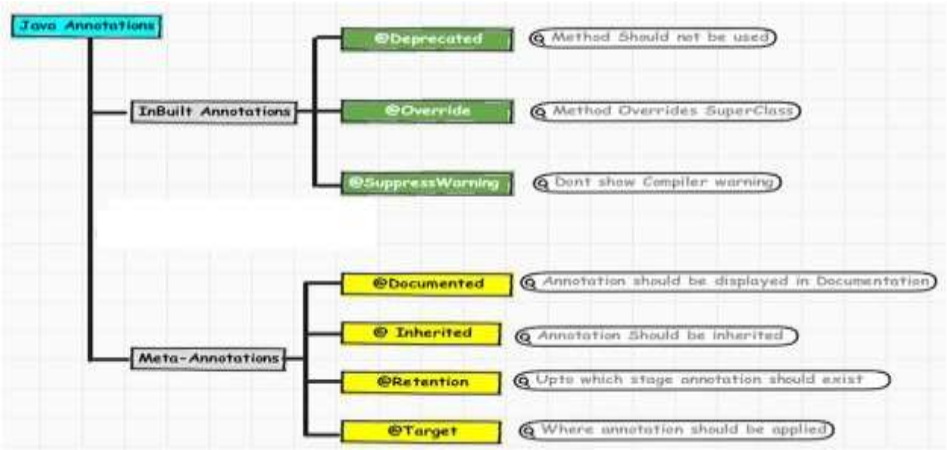
Simple annotations or In Build Annotations: These types of annotations are used to annotate the code only. We can not use these types of annotations for creating the custom annotation type.

JDK 5 includes three types of simple annotations.

@Deprecated
@Override
@SuppressWarnings

Each of these annotations are explained in the following sections.

Meta annotations: Java allows us to define our own annotation. The annotations which are used to define custom annotations are known as “Meta Annotation“ and they are also known as annotations of annotations.



15.3.4 Exploring Simple Annotations

Let us understand about Simple Annotations in detail:

@Deprecated:

The @Deprecated annotation is used to mark a class, method or field as deprecated, meaning it should no longer be used. If your code uses deprecated classes, methods or fields, the compiler will give you a warning. Here is an example:

```
@Deprecated
public class MyComponent {

}
```

The use of the @Deprecated annotation above the class declaration marks the class as deprecated.
You can also use the @Deprecated annotation above method and field declarations, to mark the method or field as deprecated.

Example:

```
public class Deprecated_method{

    @Deprecated
              public void showSomething() {

             System.out.println("Method has been depricated'");

    }
    }
```

Now lets try to invoke this method from inside the other class:

```
public class Test_Deprication {
public static void main(String arg[]) throws
Exception {
new Test_Deprication();
}
public Test_Deprication() {
Deprecated_method d = new Deprecated_method();
d.showSomething();
}
}
```

The method showSomething() in the above example is declared as the deprecated method. That means we can't further use this method any more. On compiling the class Depricated_method does not generate any error. While compiling the class Test_Deprication generates the message like this:

Note: Test_Deprication.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

Adding the -Xlint to the compilation line shows specifically what is wrong:

```
> javac -Xlint:deprecation Test_Deprication.java


Test_Deprication.java:8: warning: [deprecation] showSomething() in
  Deprecated_method has been deprecated
        d.showSomething();
           ^
  1 warning
```

Note:A Deprecated method can be used by definition. However it is recommended not to use deprecated method.

@Override

When we want to override a method, we can use this annotation to say to the compiler we are overriding an existing method. If the compiler finds that there is no matching method found in super class then generates a warning. This is not mandatory to use @Override when we override a method It is a good idea to use it still, though.

Here is an example use of the @Override annotation:

```java
public class MySuperClass {

    public void doTheThing() {
        System.out.println("Do the thing");
    }
}
 class MySubClass extends MySuperClass{

    @Override
    public void doTheThing() {
        System.out.println("Do it differently");
    }
}
```

In case the method doTheThing() in MySuperClass changes signature so that the same method in the super class no longer overrides it, the compiler will generate an error.

Example:

```
public class Animal {

    public void makeSound(){

    }
}

class Cat extends Animal{

    @Override
    public void makeSound(){
        System.out.println("myyyyyaaawwwwww");
    }
}
```

When we use @Override annotation to a method, it let compiler know that we are overriding it from super class. It is not mandatory to use @Override when you override a method. Incase you are using @Override annotation, and the method signature is not found at super class will results compilation error. Thus it helps to improve the readability of the code. It also helps in avoiding runtime errors. That is why use of this annotation is encouraged.

@SuppressWarnings

The @SuppressWarnings annotation makes the compiler suppress warnings for a given method. For instance, if a method calls a deprecated method, or makes an insecure type cast, the compiler may generate a warning. You can suppress these warnings by annotating the method containing the code with the @SuppressWarnings annotation.

This annotation type must be used with one or more warnings as its arguments, for example:

```
@SuppressWarnings("unchecked")
@SuppressWarnings({"unchecked", "deprecation"})
```

Here is an example:

```java
package com.tcs.ilp.example.annotation;

import java.util.Date;

public class SuppressWarningsExample {
    @SuppressWarnings(value={"deprecation"})
    public static void main(String[] args) {
        Date date = new Date(2008, 9, 30);

        System.out.println("date = " + date);
    }
}
```

In the example above if we don't use @SuppressWarnings annotation the compiler will report that the constructor of the Date class called above has been deprecated.

Example:

```java
package com.tcs.ilp.example.annotation;
public class MyDeprecatedExmp {

    @Deprecated
    public void showDeprecatedMessage(){
        System.out.println("This method is marked as deprecated");
    }

    @SuppressWarnings("deprecation")
    public static void main(String a[]){

        MyDeprecatedExmp mde = new MyDeprecatedExmp();
        mde.showDeprecatedMessage();
    }
}
```

This example is suppressing the deprecation warnings that means we can't see the warnings any more.

15.3.5 Meta Annotations

Annotations itself is meta information then what is meta annotations? As you have rightly guessed, it is information about annotation. When we annotate a annotation type then it is called meta annotation.

There are four types of Meta annotations (or annotations of annotations) defined by the JDK 5. These are as follows:

- **Target:**The @Target annotation specifies for what elements of the class the annotation is to be applied. Following are the possible types

    1. *@Target(ElementType.TYPE): If the value is TYPE, the annotation can be applied to any element of the class. It is the default also.*

    2. *@Target(ElementType.FIELD): Applicable to fields (instance variables) of a class*

    3. *@Target(ElementType.METHOD): Applicable to methods of a class.*

    4. *@Target(ElementType.PARAMETER): Applicable to parameters of the method.*

    5. *@Target(ElementType.CONSTRUCTOR): Applicable to a constructors.*

    6. *@Target(ElementType.LOCAL_VARIABLE): Applicable to a local variables.*

7.@Target(ElementType.ANNOTATION_TYPE):Says that declared type by itself is an annotation type

- **Retention:**The @Retention annotation describes the retention (availability or preservation) policy of the annotation. Retention policy specifies to what extent the annotation should be made available – in source code only or class file or runtime.

Following are the possible values for @Retention:

1. RetentionPolicy.SOURCE: Annotation is available in source code only but discarded while compilation.

2. RetentionPolicy.CLASS: Annotation exists in the .class file generated but discarded at runtime. As the annotation is not available at runtime, runtime cannot make use of the annotation.

3. RetentionPolicy.RUNTIME: Annotation is available for both compiler and runtime also.

That is, annotation is stored in .class file and also runtime can make use of. It is the most used.

Documented: The elements annotated with @Documented should be documented using JavaDoc tool. This annotation instructs JavaDoc tool that the annotation should be visible in Java documentation for all the classes using the custom annotation.

- **Inherited:**The element annotated with @Inherited indicates that the annotation can be inherited from any of the super classes. The compiler queries all the super classes one-by-one until the required annotation is found. This annotation is applicable to classes only.

  In the super class, if a method is annotated, the subclass overridden is not annotated. That is, annotations are not inherited to subclasses. If declared @Inherited, the subclass gets the annotation properties inherited from the super class.

Let us see one simple example to understand Inherited Meta Annotation:

```
package com.tcs.ilp.Annotations.Meta;

import java.lang.annotation.Inherited;

@Inherited
@interface Maharashtra {
}

@interface Gujarat {

}

@Maharashtra
@Gujarat
class Gujrat_Maharashtra {

}

class Nag extends Gujrat_Maharashtra {

    public void display() {

        System.out.println("Nagpur is in Maharashtra");

    }

}
```

In above example, there are two annotations defined: “@Maharashtra” and “@Gujrat”.Subclass “Nag” inherited annotation “@Maharashtra” only from its parent class “Gujarat_Maharashtra”, because it is marked as @Inherited.

Check Your Self:

1.Was there any annotation or similar functionality existing prior to 1.5 of Java ?
A.Yes B.No

2.Which of the following symbol is used to represent an annotation ?
A.# B.@ C.$ D.&

3.Which of the following way is a valid annotation definition ?

A.public @annotation MyAnnotation{ }
B.private @interface MyAnnotation{ }
C.public @interface MyAnnotation{ }
D.public @MyAnnotation{ }

4.Which of the following is a benefit from using @override annotation ?

A.Override annotation says that the method marked must be overriden by subclass
B.It helps in runtime to dynamic binding the method to appropriate object
C.Compiler can warn you if the method desingated with override is not a correct override

5.An annotation, in the Java computer programming language, is a special form of syntactic _____ that can be added to Java source code.

A. Ontology (information science)
B. Object-oriented programming
C. Metadata
D. Data model

Additional Information:

- Exploring Meta Annotations in detail

Best Practises

- Do not over use annotation as it will pollute the code.
- It is better not to try to change the behavior of objects using annotations. There is sufficient constructs available in oops and annotation is not a better mechanism to deal with it.
- **We should not what we are parsing**. Do not try to over generalize as it may complicate the underlying code. Code is the real program and annotation is meta.
- Avoid using annotation to specify environment / application / database related information.

Related Topics:

Meta Annotations Types

References:

http://docs.oracle.com/javase/tutorial/java/annotations/

http://en.wikipedia.org/wiki/Java_annotation

http://en.wikibooks.org/wiki/Java_Programming/Annotations/Introduction

Books for your References:

Thinking in Java by Bruce Eckel
Head First Java by Bert Bates, Kathy Sierra
Java 7 Programming Black Book by Dreamtech Pres